ALGORITMIA · PRÁCTICA 1 · P1.3

Los desastres

Un vector, una estructura auxiliar y un solo recorrido. Cómo pensar el ejercicio antes de escribirlo.

Qué nos piden

i	FECHA	VÍCTIMAS	ÚLTIMO
0	19900512	150 000	?
1	19950131	300 000	?
2	20011203	20 000	?
3	20020606	140 000	?
4	20070322	60 000	?
5	20130728	250 000	?

Para cada desastre, rellenar **último**: la fecha del **último desastre anterior** que tuvo **más víctimas** que él.

Si no hay ninguno, `0`.

Los datos llegan **en orden cronológico**, del más antiguo al más reciente. El enunciado pide coste `O(N)`: **un solo recorrido** del vector.

Primero, a mano

i	fecha	víctimas	último
0	19900512	150 000	0
1	19950131	300 000	0
2	20011203	20 000	19950131
3	20020606	140 000	19950131
4	20070322	60 000	20020606
5	20130728	250 000	**19950131**

El 5 tiene 250 000. Hacia atrás: **60 000**, no. **140 000**, no. **20 000**, no. **300 000**, sí: 19950131.

Mirar hacia atrás hasta encontrar uno mayor. Funciona… pero en el peor caso cada desastre repasa todos los anteriores: **N^2**. Nos piden **un solo recorrido**.

La observación que lo resuelve

i	fecha	víctimas	último
0	19900512	150 000	0
1	19950131	300 000	0
2	20011203	20 000	19950131
3	20020606	140 000	?
4	20070322	60 000	?
5	20130728	250 000	?

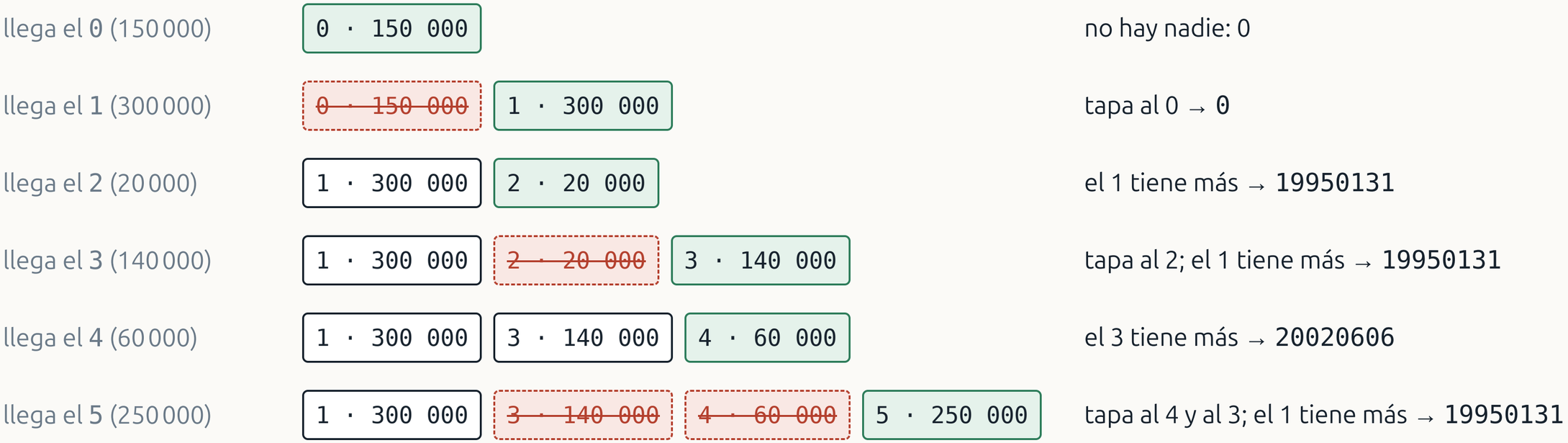
Acaba de llegar el 3, con `140 000`. Es **más reciente** que el 2 y tiene **más víctimas** que el 2.

¿Puede el 2 (`20 000`) ser la respuesta de **algún desastre futuro**? **No.** Quien mirase hacia atrás encontraría antes al 3, que le gana en las dos cosas.

Cuando llega uno con más víctimas, los anteriores con menos dejan de importar. Para siempre. Podemos olvidarlos.

Los candidatos

Lo que queda tras cada desastre: los que todavía pueden ser respuesta de alguien.



El nuevo **elimina** candidatos empezando por el **más reciente**, y su respuesta es el primero que sobrevive: también el **más reciente**. **Siempre trabajamos por el extremo más reciente.**

¿Pila o cola?

Cola

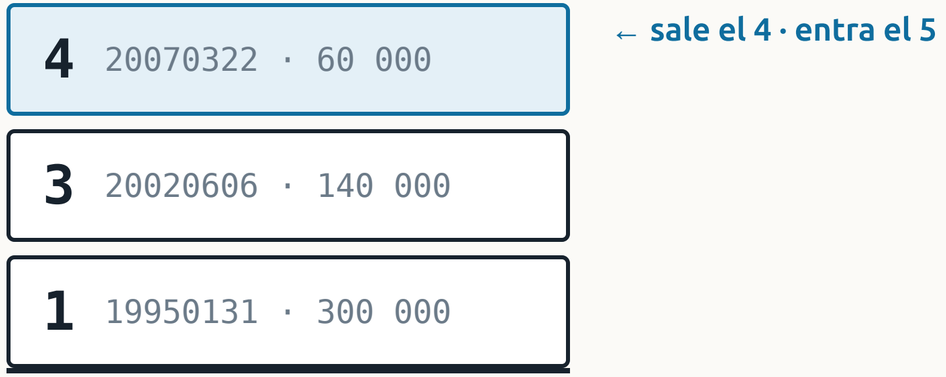
Entra por un extremo, sale por el otro. Lo que sale es **lo más antiguo**.



Nos daría el candidato **más antiguo**. **No es el que queremos.**

Pila

Entra y sale por el mismo extremo. Lo que sale es **lo más reciente**.



Nos da el candidato **más reciente**. **Es justo lo que hace falta.**

¿Qué guardo en la pila?

La pila de la práctica es de enteros. ¿Qué entero?

`la fecha`	Para responder necesito la fecha… pero para **comparar** necesito las víctimas. **No puedo comparar.**
`las víctimas`	Puedo comparar… pero cuando encuentro la respuesta **no sé qué fecha escribir.**
`el índice i`	Con `i` tengo `vector[i].fecha` **y** `vector[i].victimas`. **Todo.**

La pila guarda posiciones del vector. Es un truco que vais a reutilizar muchas veces: cuando la estructura sólo admite enteros, guarda el índice y consulta el vector.

El algoritmo, en palabras

Para cada desastre i, en orden:

1. **Mientras** la pila no esté vacía **y** la cima tenga **menos o igual** víctimas que `i`: desapilar. (quedan tapados)

2. **Si** la pila se ha quedado vacía, `último = 0`. **Si no**, `último` = la fecha de la cima.

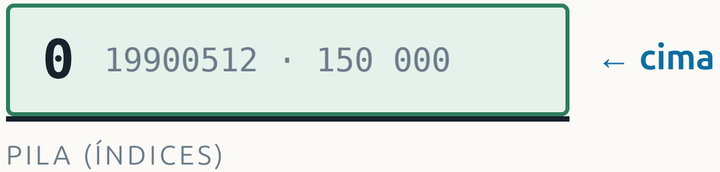
3. **Apilar** `i`: desde ahora es candidato.

> **Trampa del paso 1:** el orden de las dos condiciones importa. Consultar la cima de una pila vacía es un error; hay que preguntar **primero** si está vacía. En C, && no evalúa la segunda condición si la primera ya es falsa.

Iteración `i = 0` — 19900512, 150 000 víctimas

I	FECHA	VÍCTIMAS	ÚLTIMO
0	19900512	150 000	0
1	19950131	300 000	?
2	20011203	20 000	?
3	20020606	140 000	?
4	20070322	60 000	?
5	20130728	250 000	?

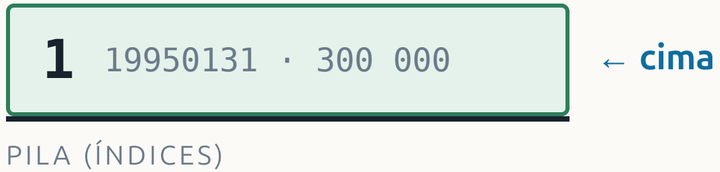
Paso 3: **apilamos el 0**. Es candidato para los que vengan.



Iteración `i = 1` — 19950131, 300 000 víctimas

i	FECHA	VÍCTIMAS	ÚLTIMO
0	19900512	150 000	0
1	19950131	300 000	0
2	20011203	20 000	?
3	20020606	140 000	?
4	20070322	60 000	?
5	20130728	250 000	?

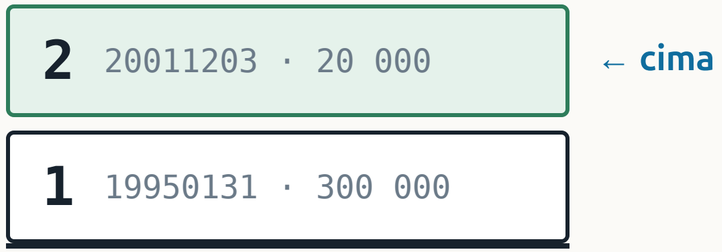
Apilamos el 1. Fíjate: el 0 ya no está en ningún sitio, y no hace falta.



Iteración `i = 2` — 20011203, 20 000 víctimas

i	FECHA	VÍCTIMAS	ÚLTIMO
0	19900512	150 000	0
1	19950131	300 000	0
2	20011203	20 000	19950131
3	20020606	140 000	?
4	20070322	60 000	?
5	20130728	250 000	?

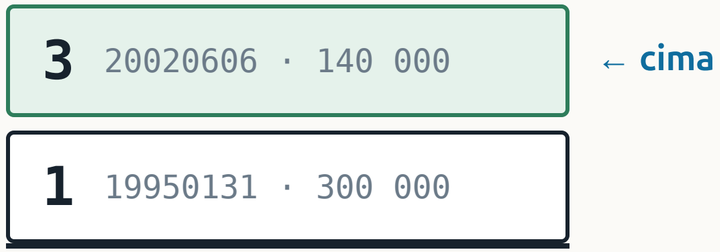
Apilamos el 2. De abajo arriba, las víctimas **van bajando**: 300 000, 20 000. Siempre será así.



Iteración `i = 3` — 20020606, 140 000 víctimas

i	FECHA	VÍCTIMAS	ÚLTIMO
0	19900512	150 000	0
1	19950131	300 000	0
2	20011203	20 000	19950131
3	20020606	140 000	19950131
4	20070322	60 000	?
5	20130728	250 000	?

Apilamos el 3. Una iteración puede desapilar **varios**, **uno** o **ninguno**: por eso es un `while` y no un `if`.

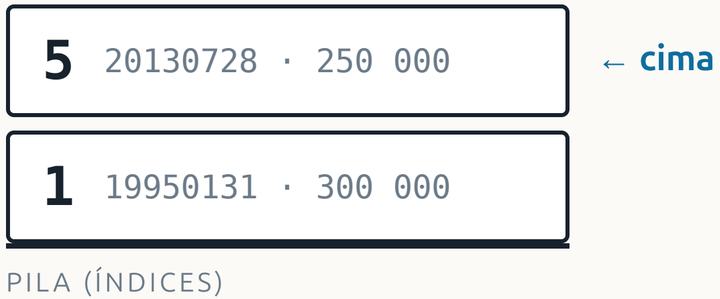


El resto sale solo. ¿Y el coste?

i	FECHA	VÍCTIMAS	ÚLTIMO
0	19900512	150 000	0
1	19950131	300 000	0
2	20011203	20 000	19950131
3	20020606	140 000	19950131
4	20070322	60 000	20020606
5	20130728	250 000	19950131

Cada índice **entra en la pila una vez** y **sale como mucho una vez**. En total, como mucho 2N operaciones de pila.

Aunque una iteración desapile muchos, **el total es O(N)**: lo que desapila hoy lo apiló otra iteración anterior.



Para escribirlo

Lo que tiene que aparecer en vuestro .c, en este orden.

- Un `struct` con los tres campos, y un vector de ellos.
- Una pila **de enteros**, creada con `nuevaPila` antes del bucle.
- Un **único** bucle sobre el vector, que lee el registro y lo procesa en la misma vuelta.
- Dentro: el `while` (¿vacía? **antes** de mirar la cima), el `if` del 0, y el `apilar(i)`.
- Comprobadlo con el ejemplo del enunciado: `0, 0, 19950131, 19950131, 20020606, 19950131`.

Y antes de teclear: **haced la simulación en papel** con vuestros propios datos, como la de hoy. Si la pila hace lo que esperáis en papel, el código sale casi solo.